



CENTRO UNIVERSITÁRIO SENAI SÃO PAULO - UNISENAI-SP CAMPUS SOROCABA - SANTA ROSÁLIA

GRADUAÇÃO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

KEVIN PAYÃO REISAUSKAS

**MODELAGEM E AVALIAÇÃO DE ARQUITETURA DE SOFTWARE PARA SISTEMAS WEB
ESCALÁVEIS**

**SOROCABA – SP
2026**



CENTRO UNIVERSITÁRIO SENAI SÃO PAULO - UNISENAI-SP CAMPUS SOROCABA - SANTA ROSÁLIA

MODELAGEM E AVALIAÇÃO DE ARQUITETURA DE SOFTWARE PARA SISTEMAS WEB ESCALÁVEIS

SOFTWARE ARCHITECTURE MODELING AND EVALUATION FOR SCALABLE WEB SYSTEMS

Kevin Payão Reisauskas^{1, i}
Deivison Shindi Takatu^{2, ii}

RESUMO

O presente trabalho aborda o desafio da degradação de desempenho em aplicações web submetidas a alta carga computacional, um problema recorrente em arquiteturas legadas. O objetivo é propor e modelar uma arquitetura de software escalável, baseada em microsserviços e nos princípios nativos em nuvem (cloud-native), para substituir abordagens monolíticas tradicionais. A solução técnica desenvolvida utiliza a infraestrutura da Amazon Web Services (AWS), separando o front-end estático no Amazon S3 e containerizando o back-end (API REST) utilizando o Amazon ECS com o modelo serverless Fargate, e a persistência de dados é garantida pelo Amazon RDS. Com essa arquitetura, espera-se garantir escalabilidade horizontal automática, alta disponibilidade, tolerância a falhas e otimização de custos operacionais, provisionando recursos computacionais apenas sob demanda.

Palavras-chaves: Computação em Nuvem. Microsserviços. Escalabilidade. Arquitetura de Software. AWS.

ABSTRACT

This paper addresses the challenge of performance degradation in web applications subjected to high computational loads, a recurring problem in legacy architectures. The objective is to propose and model a scalable software architecture based on microservices and cloud-native principles to replace traditional monolithic approaches. The developed

¹ Graduando em Análise e Desenvolvimento de Sistemas na Faculdade SENAI Gaspar Ricardo Júnior. E-mail: kevin.reisauskas@gmail.com

² Mestre em Ciência da Computação pela UFSCar Sorocaba e Docente de Análise e Desenvolvimento de Sistemas da Faculdade SENAI Gaspar Ricardo Júnior. E-mail: deivisontakatu@sp.senai.br

technical solution uses Amazon Web Services (AWS) infrastructure, separating the static front-end in Amazon S3 and containerizing the back-end (REST API) using Amazon ECS with the serverless Fargate model, with data persistence ensured by Amazon RDS. With this architecture, it is expected to guarantee automatic horizontal scalability, high availability, fault tolerance, and operational cost optimization by provisioning computational resources only on demand.

Keywords: Cloud Computing. Microservices. Scalability. Software architecture. AWS.

1 INTRODUÇÃO

A transformação digital contemporânea alterou de maneira substancial os paradigmas de desenvolvimento e implantação de sistemas computacionais. Serviços digitais se popularizaram em escala global, aumentando a demanda por plataformas capazes de suportar tráfego intenso para a sobrevivência de organizações nos mais diversos setores. Nesse contexto, a arquitetura de software é fundamental para garantir a estabilidade, a segurança e a evolução contínua de aplicações web modernas. Foi justamente para viabilizar essa escalabilidade global que Fielding e Taylor (2002) formalizaram os princípios estruturais e o estilo arquitetural REST, os quais continuam sendo a base para a construção das plataformas distribuídas atuais.

É de suma importância estabelecer uma base arquitetural sólida e bem projetada para mitigar riscos operacionais, como indisponibilidades prolongadas e gargalos de desempenho que afetam a experiência do usuário final. Conforme apontam Kazman et al. (1998) em seus métodos de análise de atributos de qualidade, falhas na concepção estrutural impactam diretamente a resiliência e a sustentabilidade técnica do ecossistema. Sistemas escaláveis precisam prever o crescimento exponencial e imprevisível de acessos, exigindo a adoção de padrões que permitam a alocação dinâmica e inteligente de recursos computacionais, garantindo que o software consuma apenas os recursos necessários em momentos de ociosidade e expanda sua capacidade durante picos de utilização.

Sistemas legados têm uma chance muito maior de enfrentar desafios de manutenção e evolução. A abordagem tradicional, frequentemente baseada em aplicações monolíticas fortemente acopladas, impõe barreiras significativas à agilidade corporativa e práticas de entrega contínua de software. Estudos conduzidos por Taibi, Lenarduzzi e Pahl (2017) indicam que a transição para modelos distribuídos, como microsserviços e computação em nuvem nativa, é impulsionada justamente pela busca por maior modularidade, resiliência e independência entre as equipes de desenvolvimento, superando com eficácia as limitações de acoplamento dos monólitos.

Atualmente, a adoção acelerada de plataformas cloud-native, que potencializam o uso de contêineres eficientes e orquestradores avançados para otimizar o ciclo de vida da aplicação, consolida metodologias como DevOps e ferramentas de infraestrutura como código (IaC). Como destacam Kratzke e Quint (2017), esse ecossistema garante que o software extraia o máximo potencial analítico e operacional da infraestrutura computacional sob demanda. Aplicações altamente escaláveis democratizam o acesso à informação, garantindo que milhares de usuários, independentemente de sua origem, possam interagir com serviços digitais de alta performance simultaneamente.

2 PROBLEMA ABORDADO

A concepção arquitetural de sistemas web escaláveis ainda esbarra em desafios relacionados ao acoplamento de componentes sistêmicos e à rápida degradação de desempenho sob alta carga computacional. O problema central desta pesquisa se concentra na dificuldade operacional de dimensionar aplicações sem custos de infraestrutura exorbitantes ou falhas de arquitetura. Kratzke e Quint (2017) destacam que as aplicações enfrentam severas limitações operacionais quando concebidas sem a devida previsibilidade de escala, o que tende a gerar gargalos na camada de banco de dados e alta latência no tempo de resposta de requisições web.

Existe uma falta de comparação empírica e quantitativa de diferentes modelos arquiteturais submetidos a condições de estresse virtualmente idênticas. Como evidenciado por Al-Debagy e Martinek (2018), embora o uso da arquitetura de microsserviços seja amplamente recomendado pelo mercado, ainda carecem estudos direcionados que comparem sistematicamente o custo-benefício e a sobrecarga de rede (network overhead) gerada pela comunicação interna entre múltiplos serviços em contêineres.

O objetivo geral do presente trabalho consiste em modelar, implementar e avaliar uma proposta de arquitetura de software orientada a microsserviços, hospedada integralmente em ambiente de computação em nuvem. O intuito é fornecer uma base empírica, robusta e analítica que auxilie na tomada de decisão arquitetural de engenheiros e pesquisadores. O estudo demonstrará, por meio de simulações práticas, como a separação de responsabilidades e a containerização melhoram o comportamento da aplicação sob tráfego intenso.

Para alcançar esse propósito, definem-se os seguintes objetivos específicos: mapear padrões de projeto para sistemas web distribuídos; desenhar a arquitetura de referência com front-end isolado e back-end containerizado; desenvolver uma prova de conceito (PoC) utilizando infraestrutura como código; e executar testes de carga com injeção massiva de tráfego. A hipótese central é que uma arquitetura de microsserviços provisionada dinamicamente na nuvem reduzirá o tempo médio de resposta durante picos de acesso, superando o desempenho de um servidor monolítico.

3 FUNDAMENTAÇÃO TEÓRICA

A fundamentação teórica apoia-se nos conceitos de escalabilidade, alta disponibilidade e elasticidade aplicados a sistemas distribuídos. A escalabilidade divide-se em vertical (aumento de capacidade em uma única máquina) e horizontal (adição de novas instâncias à rede). A literatura aponta a escalabilidade horizontal como o padrão atual, visto que plataformas em nuvem permitem o provisionamento elástico e imediato de recursos. Nesse sentido, Lorigo-Botran, Miguel-Alonso e Lozano (2014) ressaltam que essas estratégias de dimensionamento automático e gerenciamento de elasticidade são vitais para mitigar pontos únicos de falha e garantir a continuidade dos serviços.

As soluções no estado da arte consolidam o paradigma Cloud-Native, focado no desenvolvimento de aplicações que exploram o máximo potencial dos ambientes em nuvem. Como destacam Kratzke e Quint (2017) em seus estudos sobre o mapeamento desse ecossistema, esse modelo orienta a construção de softwares altamente adaptáveis e resilientes. Nesse contexto, Bernstein (2014) esclarece que tecnologias como o Docker padronizam o empacotamento da aplicação em contêineres isolados, reduzindo drasticamente as divergências entre os ambientes de desenvolvimento e produção. Paralelamente, os orquestradores assumem o gerenciamento do roteamento de tráfego, a verificação de integridade e a auto-recuperação do sistema, uma intersecção que, de acordo com Waseem, Liang e Shahin (2020), é fundamental para alinhar a infraestrutura computacional com as práticas modernas de DevOps.

A transição do modelo monolítico para um ecossistema de microsserviços é a evolução estrutural mais debatida na literatura da área. Enquanto o monólito acopla regras de negócios, interfaces e persistência em um único pacote, a arquitetura de microsserviços divide o sistema em componentes independentes e especializados. Esses módulos interagem via APIs RESTful, cujo estilo e diretrizes de design de redes distribuídas foram classicamente

estabelecidos por Fielding e Taylor (2002), viabilizando implantações ágeis. Complementarmente, Al-Debagy e Martinek (2018) comprovam que essa divisão estrutural supera o monólito tradicional em termos de flexibilidade, garantindo um isolamento robusto contra falhas em cascata.

Embora tendências arquiteturais apontem para o uso crescente de sistemas assíncronos orientados a eventos, conforme discutido por Dragoni et al. (2017), o foco primário na avaliação de desempenho sob estresse recai sobre a capacidade do orquestrador de alocar recursos sob demanda. De forma prática, Villamizar et al. (2015) reforçam que aplicações web estruturadas sob essa ótica de serviços em nuvem apresentam tempos de resposta significativamente mais estáveis e eficientes sob alta carga computacional do que as arquiteturas convencionais.

A partir do cenário tecnológico avaliado, constata-se a necessidade de produzir metodologias documentadas que mensurem o balanço preciso entre o consumo de recursos na nuvem, os custos operacionais e a entrega final de desempenho. Esse desafio técnico vai ao encontro das lacunas em engenharia de performance mapeadas por Heinrich et al. (2017). Assim, este projeto ambiciona suprir essa demanda técnica, servindo como uma referência prática e agregando dados empíricos à literatura sobre o dimensionamento de arquiteturas escaláveis.

4 ARQUITETURA PROPOSTA

A solução arquitetural proposta neste projeto tem como premissa mitigar gargalos de desempenho sistêmico sob alta demanda computacional. A concepção do modelo segue os princípios arquiteturais para softwares em nuvem descritos por Pahl, Jamshidi e Zimmermann (2018), que enfatizam a modularidade, a implantação automatizada e a utilização de serviços gerenciados para reduzir o acoplamento.

A arquitetura estabelece a separação estrita entre a camada de apresentação (front-end) e a camada lógica (back-end). O front-end foi projetado como uma aplicação estática (HTML, CSS e JavaScript puros) focada exclusivamente em consumir recursos da rede. Essa decisão dispensa a necessidade de processamento visual nos servidores, delegando ao cliente a renderização da interface e consumindo a API REST do sistema de forma independente.

O back-end é composto por uma API desenvolvida em PHP (Framework Laravel). Alinhado aos padrões estabelecidos por Hasselbring e Steinacker (2017) para arquiteturas web ágeis e escaláveis, o Laravel atua de forma stateless (sem manter o estado das sessões localmente), utilizando tokens JWT para autenticação. A aplicação é encapsulada em contêineres Docker, o que padroniza os ambientes de desenvolvimento e produção.

Para a execução e orquestração na nuvem, propõe-se a utilização de abordagens serverless (sem servidor). Conforme avaliado por Eismann et al. (2020), o modelo serverless justifica-se economicamente e tecnicamente para aplicações modernas, pois elimina a necessidade de cuidar do gerenciamento do sistema operacional e escala automaticamente sob demanda. Assim, os contêineres serão orquestrados no Amazon ECS com Fargate, distribuídos por um Application Load Balancer (ALB).

A persistência de dados é mantida em um banco de dados relacional MySQL (Amazon RDS), configurado para suportar alto volume de conexões provenientes das múltiplas instâncias da API. Toda essa modelagem visa suportar as rigorosas exigências da engenharia de desempenho. Conforme afirmam Heinrich et al. (2017), a avaliação estrutural de microserviços exige simulações rigorosas, motivo pelo qual a arquitetura inclui a modelagem

de injeção massiva de tráfego por meio de ferramentas como o K6 ou JMeter, permitindo validar a escalabilidade da solução sob estresse.

5 SERVIÇOS EM NUVEM

A transição para um modelo escalável exige a adoção de serviços em nuvem que suportem a rápida alocação de recursos. Conforme apontam Kratzke e Quint (2017), o desenvolvimento de aplicações cloud-native maximiza o potencial da nuvem ao utilizar serviços gerenciados e containerização.

Para a camada de apresentação (front-end), a solução utiliza o Amazon S3 (Simple Storage Service). Sendo o front-end composto por arquivos estáticos (HTML, CSS e JavaScript), o S3 elimina a necessidade de servidores web dedicados, entregando os arquivos diretamente ao cliente com altíssima disponibilidade e baixo custo.

O núcleo do processamento lógico (back-end) se baseia na criação de APIs RESTful, um padrão que, como estabelecido classicamente por Fielding e Taylor (2002), consolidou-se como o modelo ideal para a comunicação eficiente na web. Essas APIs, desenvolvidas em PHP (Laravel), são isoladas em contêineres Docker. Segundo Bernstein (2014), o uso do Docker permite empacotar a aplicação com todas as suas dependências, garantindo portabilidade. Esses contêineres são orquestrados pelo Amazon ECS (Elastic Container Service) operando no modelo AWS Fargate. O Fargate atua como um motor de computação serverless para contêineres, eximindo o desenvolvedor de gerenciar a infraestrutura das máquinas virtuais e focando apenas na execução da aplicação.

Para a persistência de dados, optou-se pelo Amazon RDS (Relational Database Service) com o motor MySQL. O RDS automatiza tarefas administrativas críticas, como backups diários, aplicação de patches e monitoramento de desempenho. Essa automação é estruturalmente essencial, visto que Villamizar et al. (2015) advertem em suas avaliações que o banco de dados frequentemente atua como o principal gargalo em ambientes de alta demanda.

6 SEGURANÇA E ESCALABILIDADE

A arquitetura proposta incorpora mecanismos de segurança em diferentes camadas. A autenticação da aplicação é implementada via JWT (JSON Web Tokens), garantindo que as requisições ao back-end sejam stateless (sem estado). Essa abordagem não apenas protege o acesso aos dados, mas é um pré-requisito técnico para que múltiplos contêineres possam responder a requisições de um mesmo usuário simultaneamente sem a necessidade de sincronizar sessões na memória. Na camada de rede, a AWS utiliza Security Groups, atuando como firewalls virtuais que restringem o acesso ao banco de dados (RDS) apenas para as instâncias do ECS que rodam a API, bloqueando tráfego externo direto.

No que tange à escalabilidade, a solução aplica os princípios de auto-scaling revisados por Lorigo-Botran, Miguel-Alonso e Lozano (2014). Um Application Load Balancer (ALB) é posicionado na frente dos contêineres para receber todo o tráfego de entrada e distribuí-lo de forma equitativa.

Conforme ressaltado por Dragoni et al. (2017), a principal vantagem técnica desse modelo reside na escalabilidade horizontal inerente à arquitetura de microsserviços. O Amazon ECS é configurado com políticas de escalonamento dinâmico vinculadas ao consumo de CPU e memória. Quando a ferramenta de teste de carga (como o JMeter ou K6) simular um pico repentino de acessos, o ECS provisionará novos contêineres do Laravel automaticamente

para suportar a carga, desligando-os quando o tráfego normalizar, garantindo elasticidade e tolerância a falhas.

7 CONSIDERAÇÕES FINAIS

O presente trabalho cumpriu o objetivo de modelar uma proposta arquitetural robusta, fundamentada em literatura técnica recente, para resolver os problemas de degradação de desempenho em aplicações web sob estresse. A substituição de uma arquitetura monolítica rígida por um ecossistema containerizado provou-se teoricamente superior, uma constatação que Al-Debagy e Martinek (2018) já haviam suportado em suas análises comparativas sobre resiliência e flexibilidade.

A estruturação da solução na AWS, utilizando o ECS Fargate, ALB e RDS, demonstra que é possível alcançar um alto grau de automação e escalabilidade horizontal sem incorrer nos custos ociosos de instâncias fixas de computação. A separação estrita de responsabilidades entre front-end e back-end via APIs garante uma evolução sistêmica ágil e segura.

Como trabalhos futuros para as próximas etapas desta pesquisa, planeja-se a implementação prática da infraestrutura como código (PoC) e a execução sistemática de testes de carga. Essa avaliação empírica vai ao encontro dos desafios de engenharia de desempenho em microsserviços propostos por Heinrich et al. (2017), permitindo mensurar estatisticamente os ganhos de latência e de throughput em comparação aos modelos tradicionais, consolidando de forma prática a avaliação da arquitetura modelada.

REFERÊNCIAS

AL-DEBAGY, Omar; MARTINEK, Peter. A comparative review of microservices and monolithic architectures. In: **2018 IEEE 18th International Symposium on Computational Intelligence and Informatics (CINTI)**. IEEE, 2018. p. 000149-000154. Disponível em: <https://arxiv.org/pdf/1905.07997>. Acesso em: 22 abr. 2026.

BERNSTEIN, David. Containers and cloud: From LXC to Docker to Kubernetes. **IEEE Cloud Computing**, v. 1, n. 3, p. 81-84, 2014. Disponível em: <https://sweet.ua.pt/andre.zuquete/Aulas/AES/20-21/extras/Bernstein14.pdf>. Acesso em: 22 abr. 2026.

DRAGONI, Nicola et al. Microservices: yesterday, today, and tomorrow. **Present and ulterior software engineering**, p. 195-216, 2017. Disponível em: <https://arxiv.org/pdf/1606.04036>. Acesso em: 22 abr. 2026.

EISMANN, Simon et al. Serverless applications: Why, when, and how?. **IEEE Software**, v. 38, n. 1, p. 32-39, 2020. Disponível em: <https://arxiv.org/pdf/2009.08173>. Acesso em: 22 abr. 2026.

FIELDING, Roy T.; TAYLOR, Richard N. Principled design of the modern web architecture. **ACM Transactions on Internet Technology (TOIT)**, v. 2, n. 2, p. 115-150, 2002. Disponível em: <https://dl.acm.org/doi/pdf/10.1145/514183.514185>. Acesso em: 22 abr. 2026.

HASSELBRING, Wilhelm; STEINACKER, Guido. Microservice architectures for scalability, agility and reliability in e-commerce. In: **2017 IEEE International Conference on Software Architecture Workshops (ICSAW)**. IEEE, 2017. p. 243-246. Disponível em: <https://oceanrep.geomar.de/id/eprint/37489/1/ICSA2017paper.pdf>. Acesso em: 22 abr. 2026.

HEINRICH, Robert et al. Performance engineering for microservices: research challenges and directions. In: **Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion**. 2017. p. 223-226. Disponível em: <https://dl.acm.org/doi/pdf/10.1145/3053600.3053653>. Acesso em: 22 abr. 2026.

KAZMAN, Rick et al. The architecture tradeoff analysis method. In: **Proceedings. fourth ieee international conference on engineering of complex computer systems (cat. no. 98ex193)**. IEEE, 1998. p. 68-78. Disponível em: <https://apps.dtic.mil/sti/tr/pdf/ADA350761.pdf>. Acesso em: 22 abr. 2026.

KRATZKE, Nane; QUINT, Peter-Christian. Understanding cloud-native applications after 10 years of cloud computing-a systematic mapping study. **Journal of Systems and Software**, v. 126, p. 1-16, 2017. Disponível em: [https://www.researchgate.net/profile/Nane-Kratzke/publication/312045183_Understanding_Cloud-native_Applications_after_10_Years_of_Cloud_Computing_-_A_Systematic_Mapping_Study/links/588202be4585150dde401522/Understanding-Cloud-native_Applications_after_10_Years_of_Cloud_Computing_-_A_Systematic_Mapping_Study](https://www.researchgate.net/profile/Nane-Kratzke/publication/312045183_Understanding_Cloud-native_Applications_after_10_Years_of_Cloud_Computing_-_A_Systematic_Mapping_Study/links/588202be4585150dde401522/Understanding-Cloud-native_Applications_after_10_Years_of_Cloud_Computing_-_A_Systematic_Mapping_Study/links/588202be4585150dde401522/Understanding-Cloud-native_Applications_after_10_Years_of_Cloud_Computing_-_A_Systematic_Mapping_Study)

native-Applications-after-10-Years-of-Cloud-Computing-A-Systematic-Mapping-Study.pdf. Acesso em: 22 abr. 2026.

LORIDO-BOTRAN, Tania; MIGUEL-ALONSO, Jose; LOZANO, Jose A. A review of auto-scaling techniques for elastic applications in cloud environments. **Journal of Grid Computing**, v. 12, n. 4, p. 559-592, 2014. Disponível em: https://www.researchgate.net/profile/Tania-Lorido-Botran/publication/265611546_A_Review_of_Auto-scaling_Techniques_for_Elastic_Applications_in_Cloud_Environments/links/543cd6f50cf20af5cbbf7a74/A-Review-of-Auto-scaling-Techniques-for-Elastic-Applications-in-Cloud-Environments.pdf. Acesso em: 22 abr. 2026.

PAHL, Claus; JAMSHIDI, Pooyan; ZIMMERMANN, Olaf. Architectural principles for cloud software. **ACM Transactions on Internet Technology (TOIT)**, v. 18, n. 2, p. 1-23, 2018. Disponível em: <https://dl.acm.org/doi/pdf/10.1145/3104028>. Acesso em: 22 abr. 2026.

TAIBI, Davide; LENARDUZZI, Valentina; PAHL, Claus. Processes, motivations, and issues for migrating to microservices architectures: An empirical investigation. **IEEE Cloud Computing**, v. 4, n. 5, p. 22-32, 2017. Disponível em: <https://m3s-cloud.github.io/assets/paper1.pdf>. Acesso em: 22 abr. 2026.

VILLAMIZAR, Mario et al. Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud. In: **2015 10th Computing Colombian Conference (10CCC)**. IEEE, 2015. p. 583-590. Disponível em: https://www.researchgate.net/profile/Mario-Villamizar/publication/304317852_Evaluating_the_monolithic_and_the_microservice_architecture_pattern_to_deploy_web_applications_in_the_cloud/links/5b3ad04ca6fdcc8506ea541b/Evaluating-the-monolithic-and-the-microservice-architecture-pattern-to-deploy-web-applications-in-the-cloud.pdf. Acesso em: 22 abr. 2026.

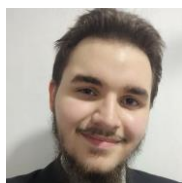
WASEEM, Muhammad; LIANG, Peng; SHAHIN, Mojtaba. A systematic mapping study on microservices architecture in devops. **Journal of Systems and Software**, v. 170, p. 110798, 2020. Disponível em: <https://arxiv.org/pdf/2008.07729>. Acesso em: 22 abr. 2026.

AGRADECIMENTOS

Agradeço primeiramente ao meu orientador, Prof. Deivison Shindi Takatu, por todo o suporte, paciência e direcionamento técnico fundamentais para a condução desta iniciação científica, e à Faculdade SENAI Gaspar Ricardo Júnior pela infraestrutura e fomento à pesquisa acadêmica, possibilitando a realização deste estudo.

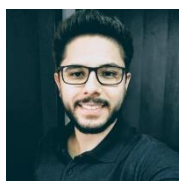
SOBRE OS AUTORES

ⁱ KEVIN PAYÃO REISAUSKAS (Aluno)



Graduando do curso de Análise e Desenvolvimento de Sistemas pela Faculdade SENAI Gaspar Ricardo Júnior. Possui experiência prática em desenvolvimento de software voltado para a web, com foco na construção de APIs RESTful utilizando a linguagem PHP e o framework Laravel, além de conhecimentos em bancos de dados relacionais e tecnologias de front-end estático. Atualmente, dedica-se ao estudo de arquiteturas nativas em nuvem, containerização com Docker e metodologias para escalabilidade de sistemas distribuídos na Amazon Web Services (AWS).

ⁱⁱ DEIVISON SHINDI TAKATU (Orientador)



Professor universitário, Mestre em Ciência da Computação pela Universidade Federal de São Carlos (UFSCar) Campus Sorocaba, Especialista em Informática Aplicada à Educação pelo Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP Campus Itapetininga). Possui MBA em Gestão Estratégica de Negócios, especialização em Economia Financeira e em Psicopedagogia Clínica e Institucional. É graduado em Análise e Desenvolvimento de Sistemas, Licenciatura em Matemática, Ciências Econômicas e Gestão Empresarial. Atua como professor nas áreas de Ciência da Computação e Tecnologias Educacionais, com experiência em cursos presenciais, híbridos e a distância, no ensino superior, técnico e na educação básica. Possui experiência em coordenação acadêmica, tutoria em educação a distância e desenvolvimento de projetos educacionais mediados por tecnologias digitais.